KUBERNETES

Kubernetes has a break-even point.

Three conditions decide whether the complexity earns its keep.

Swipe to see the line →

MILIND NAIR

What Kubernetes actually does

Scheduling

Places workloads across a fleet of nodes based on CPU, memory, and affinity rules.

Self-healing

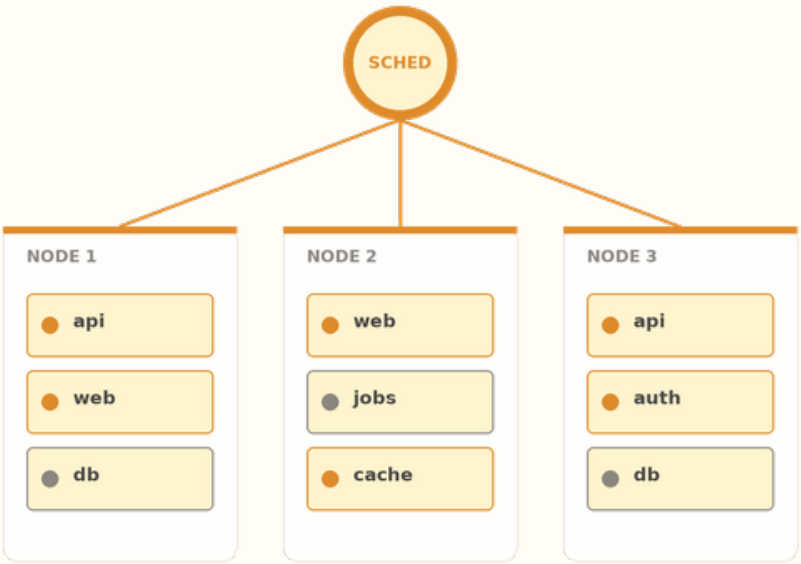
Restarts failed containers, reschedules off dead nodes, gates traffic on health checks.

Service networking

Internal DNS, load balancing, and stable addressing for pods that come and go.

Rollouts

Declarative rolling updates with automatic rollback when health checks fail.



PRIMITIVES

Four capabilities you would otherwise build and maintain yourself.

MILIND NAIR

The complexity tax

Operational surface

API server, etcd, scheduler, controller manager, kubelet, CNI, CSI, ingress. Each one fails in its own way.

Upgrade cadence

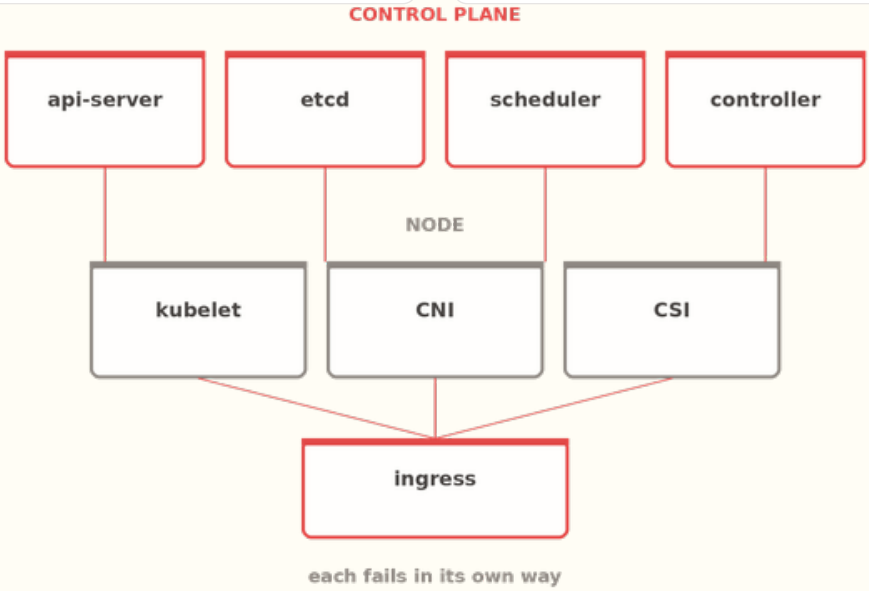
Kubernetes ships three minor versions a year, each supported for 14 months. Falling behind drops you out of the security patch window.

Abstraction debt

Every primitive has a dozen knobs you will eventually need to tune in production.

Observability cost

Cluster metrics, pod metrics, and application metrics are three separate systems you now own.



COST

The tax is paid in engineer-hours, not dollars.

MILIND NAIR

PAYS OFF

Three conditions make the cost worth paying

Multi-service footprint

You run 15 or more services that need to share infrastructure rather than each getting its own VM pool.

Uneven traffic

Load swings by 10x or more across the day, and paying for peak capacity around the clock is more expensive than autoscaling.

High deployment velocity

You ship to production several times a day across many services, and rolling updates with rollback are a first-class need.

ALL THREE

If all three hold, the control plane earns its keep.

SKIP IT

Three conditions make the cost larger than the benefit

Small service count

You run under five services. The control plane will cost more to operate than the workloads it runs.

Steady load

Traffic is flat within a 2x range. Autoscaling is the main structural benefit of Kubernetes, and flat traffic leaves it on the table.

No dedicated operator

No single engineer is the unambiguous owner of the cluster. Shared ownership of distributed systems leaves outages that no one can debug.

ANY ONE

Any one of these is enough to pick something simpler.

Three alternatives to reach for first

OPTION	BEST FOR	TRADEOFF
ECS + Fargate	**AWS shops running under 20 services that want containers without managing nodes.**	**Locks you into AWS. Per-task pricing adds up at scale.**
PaaS (Fly.io, Render)	**Product teams that want Heroku ergonomics with container flexibility.**	**Opinionated platforms. Less control in exchange for faster shipping.**
Nomad	**Teams on the HashiCorp stack that want orchestration without the Kubernetes API surface.**	**Smaller ecosystem. Fewer third-party operators and integrations.**

VERDICT

Most teams land on one of these three and never need to look further.

MILIND NAIR

Three questions before you adopt

Answer honestly. The threshold is sharper than it looks.

1. **Do you run 15 or more services in production?**

   → If no, a managed container platform will serve you better at a fraction of the operational cost.

2. **Does one engineer unambiguously own the cluster?**

   → If no, pick a platform that hides the cluster entirely. Shared ownership of Kubernetes ends in unowned incidents.

3. **Do you need features only Kubernetes offers, like custom operators or multi-tenant scheduling?**

   → If no, the ecosystem is not a reason on its own. Reach for it only when the alternatives cannot model your problem.

RULE OF THUMB

The threshold is roughly 15 services and one unambiguous cluster owner. Below either, the cost is larger than the benefit.

MILIND NAIR

The default answer is no.

Kubernetes solves scheduling, self-healing, service networking, and rollouts.
The tax is a large operational surface that simpler platforms do not carry.
Adopt when you run 15 or more services and one engineer is the unambiguous owner of t
Below that threshold, a managed platform wins on cost and reliability.

MILIND NAIR